Revision 1

Instructor: Dr. Khaled Mohamad

> **Instructions:** Answer all questions in order, write clean and well-commented code, and **use consistent Markdown formatting for both questions and answers. It is weighted at 20%.**

Dataset Overview

You will work with the **Titanic** dataset (built into Seaborn) and the **Tips** dataset (also built into Seaborn). No file uploads are needed.

In [1]:
```
#imports the necessary lib
```

Part 1: Load and Explore the Titanic Dataset

Q1. Load & Inspect

1. Load the Titanic dataset using `sns.load_dataset('titanic')` and store it in a variable called `df`.
2. Print the **shape** of the dataset.
3. Display the **first 6 rows**.
4. Print the **column names**.

ANSWER

In [2]:
```
#1)

#2)

#3)

#4)
```

Q2. Data Types & Missing Values

1. Use `.info()` to print column data types and non-null counts.
2. Print the **number of missing values per column** using `.isnull().sum()`.
3. Which column has the **most missing values**? Write your answer in a Markdown cell.

ANSWER

In [3]:
```
# 1)

# 2)
```

Q2.3.Your answer :

> ...

Q3. Basic Statistics

1. Use `.describe()` to display summary statistics for numeric columns.
2. From the output, answer in a Markdown cell:
   - What is the **mean age** of passengers?
   - What is the **maximum fare** paid?
   - What percentage of passengers survived? *(Hint: look at the `survived` mean, it represents the survival rate)*

ANSWER

In [4]:
```
# 1)
```

Answer Q3.2:

- ...
- ...
- ...

Q4. Duplicates & Categorical Summary

1. Check how many **duplicate rows** exist in the dataset.
2. Use `.describe(include=['object', 'category'])` to get a summary of categorical columns.
3. From the output, answer in a Markdown cell:
   - What is the most common **sex** in the dataset and how many rows does it appear in?
   - What is the most common **embarkation port**?

ANSWER

In [5]:
```
# 1)

# 2)
```

Answer Q4.3:

- ...
- ...

Part 2: Data Cleaning

Q5. Handle Missing Values

Create a new DataFrame called `df_clean` and apply the following cleaning steps **in order**:

1. **Drop** the `deck` column (too many missing values).
2. **Fill** missing values in the `age` column with the **median** age.
3. **Fill** missing values in the `embarked` column with the **mode** (most frequent value).
4. **Fill** missing values in the `embark_town` column with its **mode**.
5. After cleaning, print `.isnull().sum()` to confirm no missing values remain.

ANSWER

In [6]:
```
# make a copy

# Step 1: Drop deck

# Step 2: Fill age with median

# Step 3: Fill embarked with mode

# Step 4: Fill embark_town with mode

# Step 5: Confirm
```

Q6. Remove Duplicates & Redundant Columns

Continue working with `df_clean`:

1. Remove **duplicate rows** from `df_clean` using `drop_duplicates(inplace=True)`.
2. Drop the columns `'alive'` and `'embark_town'` - **REASON** - they are redundant (duplicated information from `survived` and `embarked`).
3. Print the **shape before and after** to show how many rows were removed.

4. Display the first 5 rows of the cleaned DataFrame.

ANSWER

In [7]:
```
#3) print shape before

#1)

#2)

#3) print shape after

#4) Display the first 5 rows
```

Q7. Outlier Detection

1. Create a **boxplot** for the `fare` column to visually identify outliers.
2. Use the **IQR method** to calculate the upper bound for outliers:
   - `Q1 = fare.quantile(0.25)`, `Q3 = fare.quantile(0.75)`
   - `IQR = Q3 - Q1`
   - `upper_bound = Q3 + 1.5 * IQR`
3. Print how many rows have a fare above the upper bound.
4. Create a new column `fare_capped` that **clips** the fare at the upper bound using `.clip(upper=upper_bound)`.

ANSWER

In [8]:
```
# Part 1: boxplot
```

In [9]:
```
# Part 2: IQR method
```

In [10]:
```
# Part 3: count outliers
```

In [11]:
```
# Part 4: cap fares
```

Part 3: Filtering, Sorting & Grouping

Q8. Filtering

Using `df_clean`:

1. Filter passengers who **survived** (`survived == 1`) AND were in **First class** (`class == 'First'`). How many are there?
2. Filter passengers who were **older than 60**. Print how many there are and display their names if the column exists, otherwise display the filtered DataFrame.
3. Filter passengers who paid a fare **above $50** and were **female**. How many rows match?

ANSWER

In [12]:
```
# Part 1
```

In [13]:
```
# Part 2
```

In [14]:
```
# Part 3
```

Q9. Sorting

Using `df_clean`:

1. Sort the DataFrame by `fare` in **descending order** and display the **top 5 rows** (only columns: `sex`, `age`, `class`, `fare`, `survived`).
2. Sort by `age` in **ascending order** and display the **5 youngest passengers**.
3. In a Markdown cell, state one observation from each sorted result.

ANSWER

In [15]:
```
#1)

#2)
```

Your observations:

- ...
- ...

Q10. GroupBy & Aggregation

Using `df_clean`:

1. Use `.groupby('sex')['survived'].mean()` to find the **survival rate by sex**. Round to 2 decimal places.
2. Use `.groupby('class')['survived'].mean()` to find the **survival rate by passenger class**.
3. Use `.groupby('class')['fare'].mean()` to find the **average fare per class**.
4. In a Markdown cell, write **two observations** from the results above.

ANSWER

In [16]:
```
#1)

#2)

#3)
```

Your observations:

1. ...
2. ...

Part 4: Visualization

Q11. Histogram & Count Plot

Create a figure with **two subplots side by side** (`figsize=(12, 4)`):

- **Left subplot:** Histogram of `age` using `sns.histplot` with `kde=True` and `bins=20`. Title: `'Age Distribution'`.
- **Right subplot:** Count plot of `survived` using `sns.countplot`. Title: `'Survival Count'`.

Use `plt.tight_layout()` and `plt.show()`.

ANSWER

In [ ]:

```

```

Q12. Categorical Plots

Create a figure with **two subplots side by side** (`figsize=(12, 5)`):

- **Left:** `sns.countplot` with `x='sex'` and `hue='survived'`. Title: `'Survival by Sex'`.
- **Right:** `sns.countplot` with `x='class'` and `hue='survived'`. Title: `'Survival by Class'`.

In a Markdown cell below, interpret each plot in **one sentence each**.

ANSWER

In [ ]:

```

```

Your interpretations:

- ...

- ...

Q13. Boxplot & Scatter Plot

1. Create a **boxplot** using `sns.boxplot(x='class', y='age', hue='survived', data=df_clean)`. Title: `'Age by Class and Survival'`.
2. Create a **scatter plot** using `sns.scatterplot(x='age', y='fare_capped', hue='survived', data=df_clean, alpha=0.6)`. Title: `'Age vs Fare (Capped) by Survival'`.
3. In a Markdown cell, write **one observation** from each plot.

ANSWER

In [17]:
```
# Part 1. Boxplot
```

In [18]:
```
# Part 2. Scatter plot
```

Your observations:

- ...
- ...

Q14. Correlation Heatmap

1. Select all **numeric columns** from `df_clean` using `select_dtypes(include=np.number)`.
2. Compute the **correlation matrix** using `.corr()`.
3. Plot it as a **heatmap** using `sns.heatmap(... annot=True, cmap='coolwarm', center=0)`. Set `figsize=(8, 6)`. Title: `'Correlation Heatmap'`.
4. In a Markdown cell, identify the **two features most positively correlated with survival**.

ANSWER

In [19]:
```
#1)

#2)
```

Answer Q14.4: ...

Part 5: Tips Dataset: Filtering & Grouping

Q15. Load and Filter the Tips Dataset

1. Load the **tips** dataset: `tips = sns.load_dataset('tips')`.
2. Display the first 5 rows and print the shape.
3. Filter only **dinner** orders where the **total bill is above $20**. How many rows match?
4. From those filtered rows, find the **average tip**.

ANSWER

In [21]:
```
# 1)

# 2)

# 3)

# 4)
```

Q16. Grouping the Tips Dataset

Using the full `tips` DataFrame:

1. Use `.groupby('day')['tip'].mean()` to find the **average tip by day**. Sort from highest to lowest.
2. Use `.groupby('smoker')['tip'].mean()` to compare tips between **smokers and non-smokers**.
3. In a Markdown cell, answer: **Do smokers tip more or less on average?**

ANSWER

In [22]:
```
#1)

#2)
```

Answer Q16.3: ...

Q17. Visualize the Tips Dataset

Create a figure with **three subplots** (`figsize=(15, 4)`):

- **Left:** Scatter plot of `total_bill` vs `tip` (use `plt.scatter`). Title: `'Tip vs Total Bill'` .
- **Center:** Bar plot of average tip by day (use `.plot(kind='bar')` on the grouped result). Title: `'Average Tip by Day'` .
- **Right:** Histogram of `tip` with `bins=15` and `edgecolor='k'` . Title: `'Tip Distribution'` .

Add axis labels and `plt.tight_layout()` .

ANSWER

In [ ]:
```

```

Linear Regression on Tips

Using the `tips` dataset:

1. Extract `X` = `total_bill` (reshape to column: `tips[['total_bill']].values`) and `y` = `tip` (`tips[['tip']].values`).
2. Train a `LinearRegression` model from `sklearn.linear_model`.
3. Print the **coefficient** and **intercept**. Interpret them in a sentence: "For every extra 1 dollar on the bill, the tip increases by..."
4. Plot the data as a scatter plot and overlay the **regression line** in red.
5. Predict the tip for a total bill of **$30**.

ANSWER

In [ ]:

```

```

In [23]:

```
#2) train model
```

In [24]:

```
#3) coefficient and intercept
```

In [25]:

```
# 4) scatter + regression line
```

In [26]:

```
# 5) predict for $30
```